



RiskTrace-Software Supply-Chain Risk Detector

Presented:-

Shaurya Sangwan • Abhishek Batra • Sushain Sharma
[Batch 3C32]





Problem Statement



Modern software applications rarely depend only on code written by their developers. They rely on a large number of third-party and open-source packages.



For example, an e-commerce application may use:

- React
- Express
- Axios
- JSON Web Token
- Payment libraries
- Database libraries

These dependencies can further depend on other packages.



The problem

A package in the dependency chain may:

- contain a known vulnerability
- be outdated
- be poorly maintained
- be compromised or malicious
- introduce risk through a transitive dependency

Our goal

RiskTrace analyzes a GitHub repository, identifies its dependencies and vulnerabilities, and helps developers understand which dependencies require attention first.



Project Background

A software supply chain consists of the libraries, packages, tools and dependencies used to build and run an application.

Key challenges

- **Large dependency trees** – applications can contain hundreds of direct and transitive dependencies. Malicious or compromised packages represent a separate supply-chain security signal.
- **Alert overload** – vulnerability scanners may generate many finding, Known vulnerabilities may affect only particular package versions.
- **Limited prioritization** – severity alone does not provide complete dependency context.
- **Manual investigation** – developers must inspect package versions, dependency paths and vulnerability information separately.



Proposed Solution

RiskTrace

A developer-focused platform that takes a GitHub repository and performs automated dependency-risk analysis.

What RiskTrace does

1. Repository Collection

- Accept GitHub repository URL
- Retrieve repository files and metadata

2. Dependency Analysis

- Detect dependency manifests
- Extract direct dependencies
- Resolve transitive dependencies

3. Vulnerability Detection

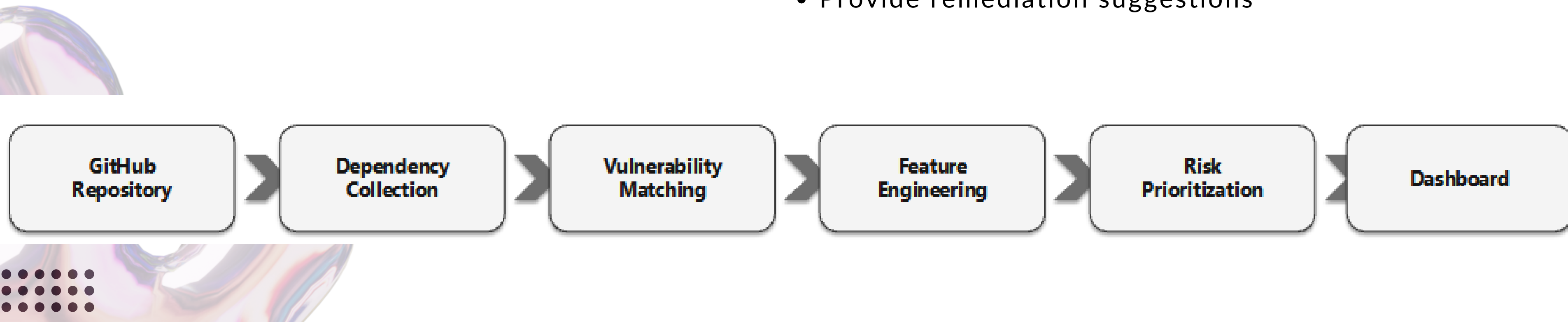
- Match package + version against vulnerability databases
- Identify affected versions
- Identify severity and fixed versions where available

4. Risk Assessment

- Combine vulnerability and dependency information
- Generate risk-related features
- Prioritize dependencies

5. Developer Dashboard

- Display security score
- Show vulnerable dependencies
- Show risk reasons
- Provide remediation suggestions



Methodology: Current Prototype Scan Pipeline

Step 1 – RepositoryCollector

- Validates the GitHub repository URL
- Fetches repository metadata using the GitHub API
- Prepares the repository for analysis
-

Step 2 – RepositoryExplorer

- Traverses the repository structure
- Searches for supported manifest files
- Skips unnecessary directories such as .git and node_modules

Step 3 – ManifestDetector

- Detects the manifest type
- Identifies the project ecosystem

Step 4 – DependencyExtractor

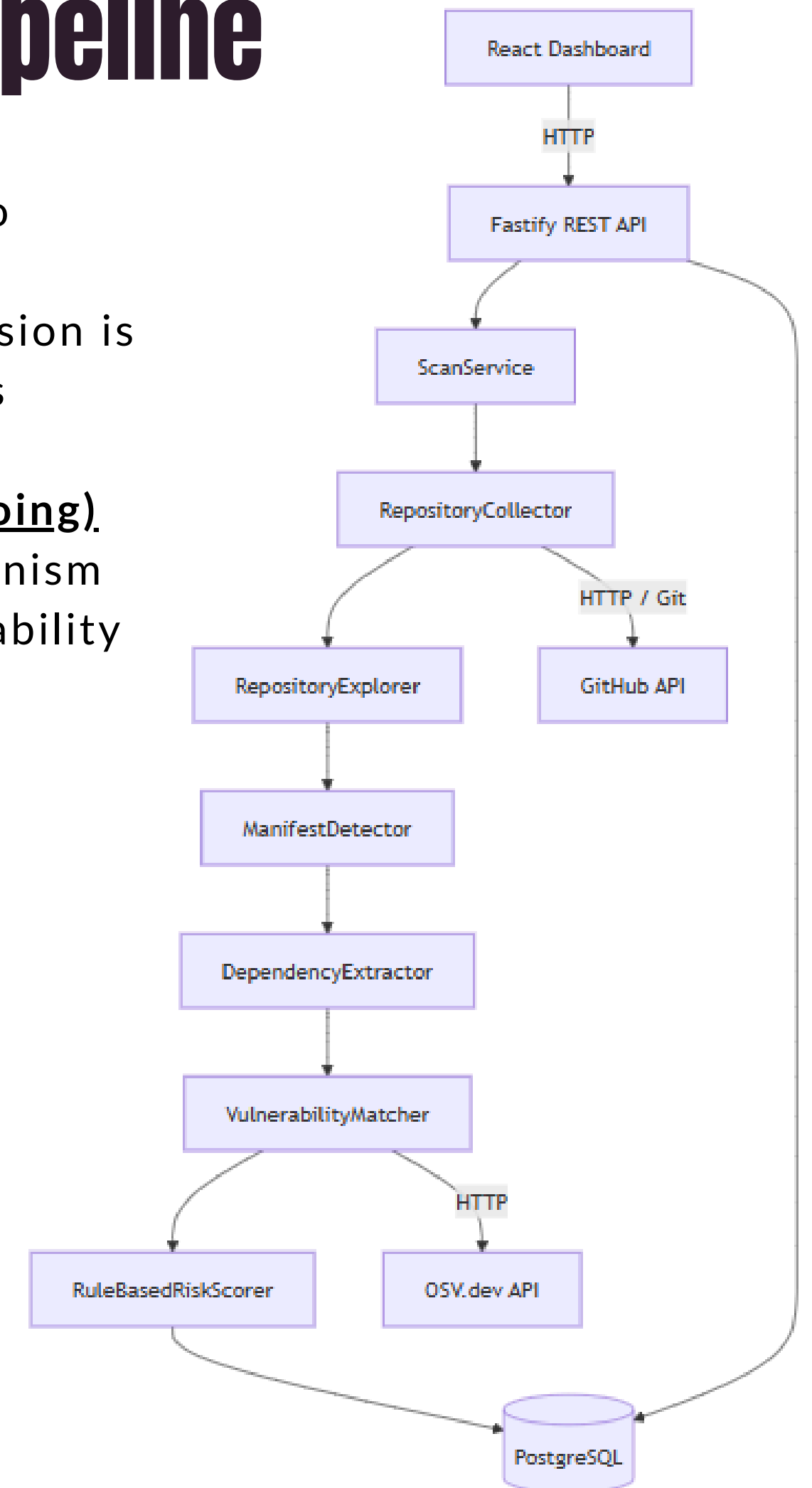
- Reads the detected manifest
- Extracts dependency names and versions
- Converts them into a normalized structure

Step 5 – VulnerabilityMatcher

- Sends dependency information to OSV.dev
- Checks whether the package/version is affected by known vulnerabilities

Step 6 – RuleBasedRiskScorer (ongoing).

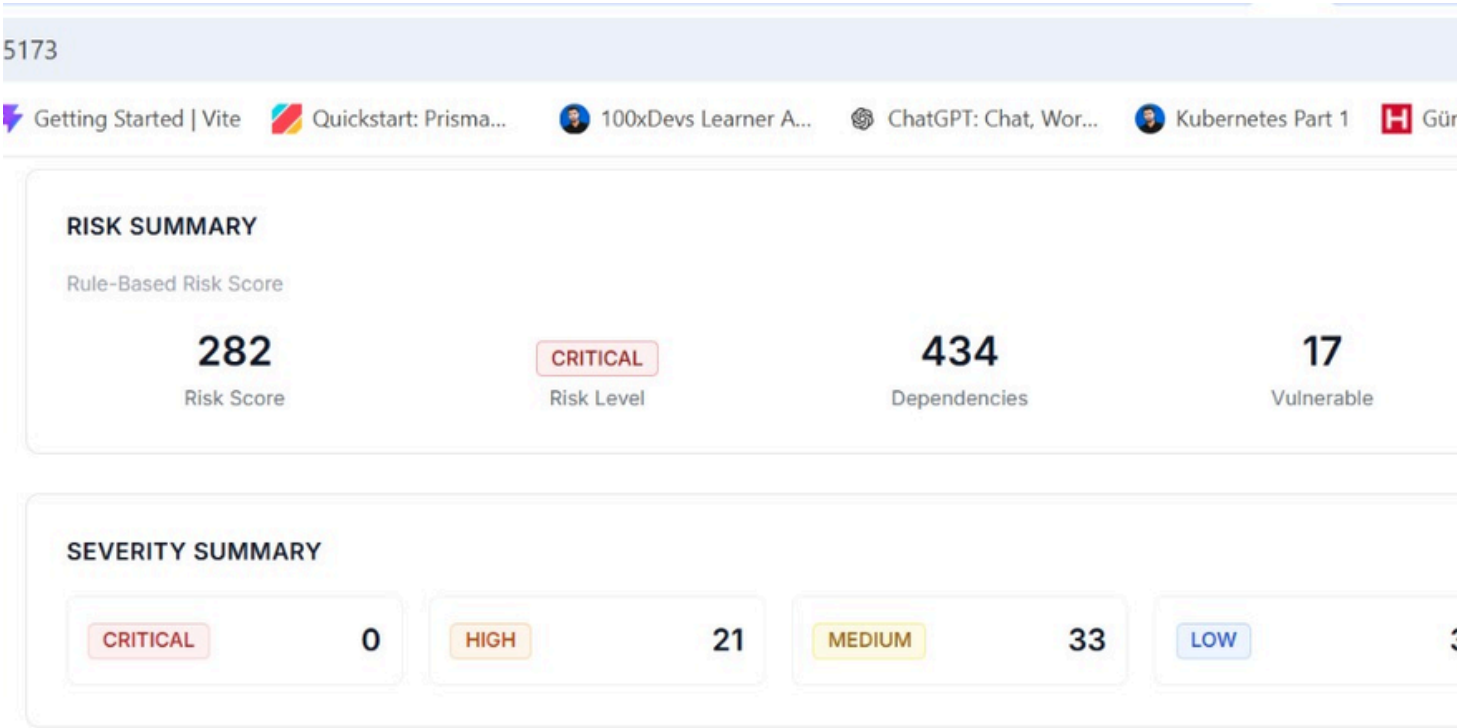
- Current prototype scoring mechanism
- Assigns weights based on vulnerability severity
- Produces a temporary risk score



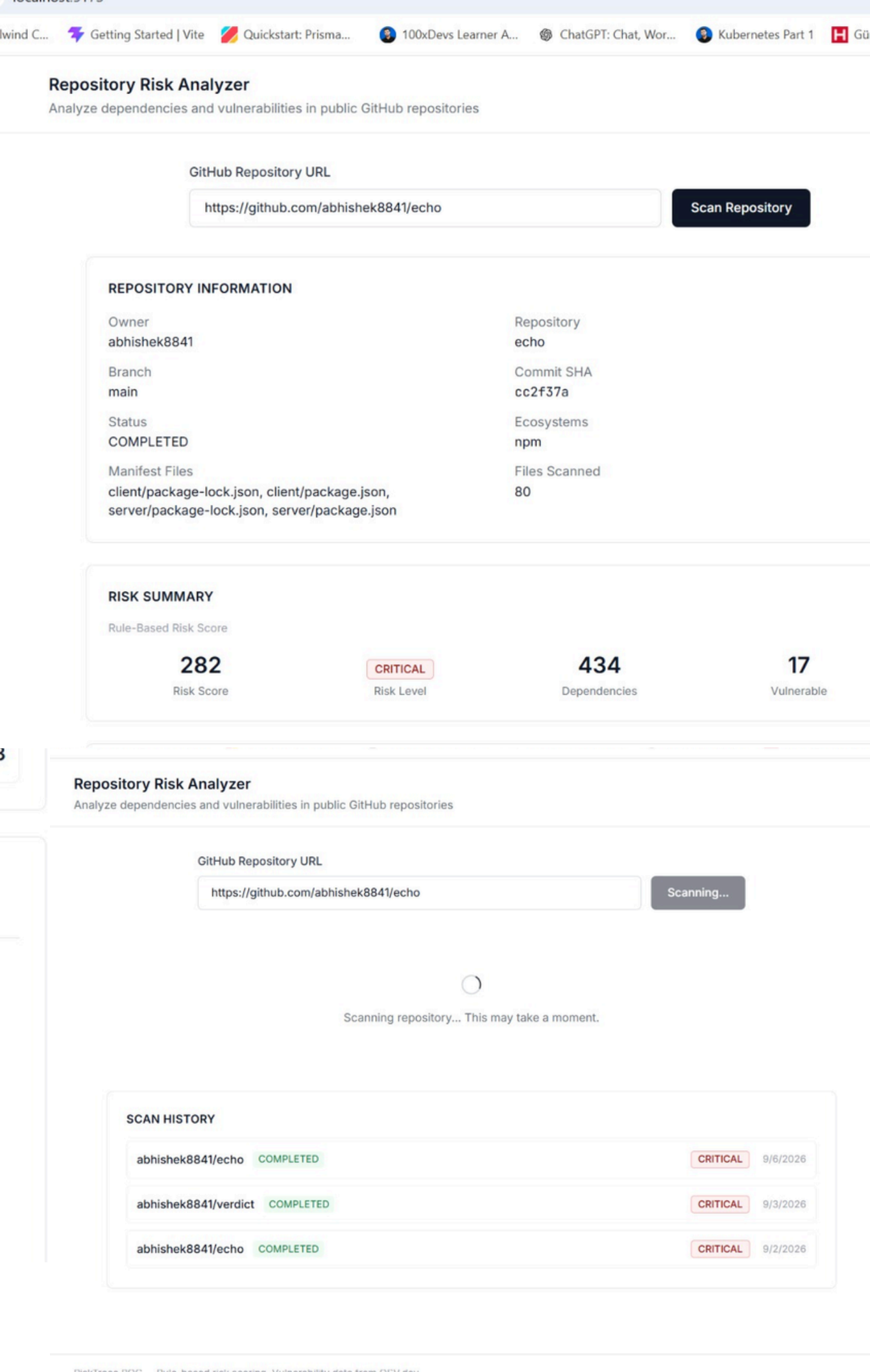
Project Result

Current prototype demonstrates

- GitHub repository input
- Repository exploration
- Manifest detection
- Dependency extraction
- OSV vulnerability matching
- PostgreSQL persistence
- Risk dashboard structure
- Rule-based risk scoring



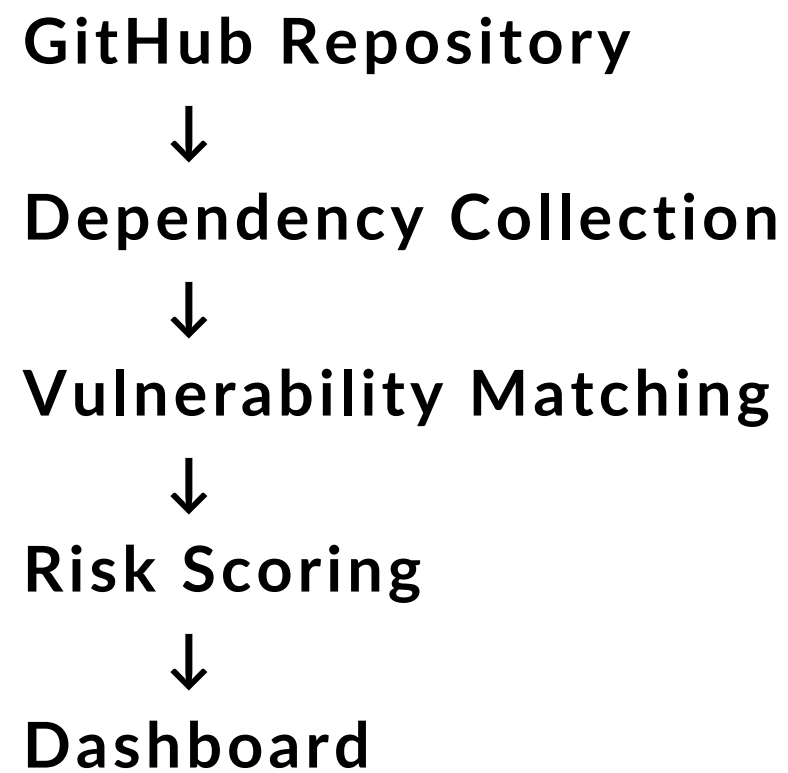
DEPENDENCIES (434)					
PACKAGE	VERSION	ECOSYSTEM	DIRECT	VULNERABILITIES	SEVERITY
hono	4.12.23	npm	Yes	15	HIGH
axios	1.17.0	npm	Yes	10	HIGH
fast-uri	3.1.2	npm	Yes	6	HIGH
react-router	7.17.0	npm	Yes	5	HIGH
brace-expansion	5.0.6	npm	Yes	3	HIGH
@hono/node-server	1.19.11	npm	Yes	2	MEDIUM





Conclusion

01 RiskTrace provides a unified workflow for analyzing software dependencies and identifying supply-chain security risks. The prototype establishes the core pipeline:



02 Next Steps:-

- Integrate the locked OSS/NVD + DataDog datasets
- Prepare ML features
- Train XGBoost
- Compare with Random Forest and Logistic Regression
- Add SHAP explanations
- Improve dependency-risk prioritization
- Add actionable remediation recommendations
- Extend dependency support and monitoring



**Thank
You**

